DÉMO 004-B · CHANGEMENTS ENTRE LES ÉTAPES

Cycle de vie et optimisations

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Étape 1** · etape-1-cycle-logs (3 fichiers, en entier)
2. **Étape 1 → 2** · etape-2-react-memo (3 fichiers)
3. **Étape 2 → 3** · etape-3-usecallback-usememo (3 fichiers)

Étape 1 etape-1-cycle-logs : point de départ

Cycle de vie : renderCount avec useRef, logs dans useEffect (Mounted / Unmounted)

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

Styles fournis (copiés tels quels, pas tapés en live) : src/App.css (48 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, eslint.config.js, public/vite.svg, package.json

Présents mais pas encore branchés (non affichés) : src/components/CarsMemo.tsx, src/components/CarsUseMemo.tsx

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 19 lignes

```jsx
import Home from "./components/Cars";
import CarsMemo from "./components/CarsMemo";
import CarsUseMemo from "./components/CarsUseMemo";

import { useState } from "react";
import "./App.css";

export default function App() {
  const [showHome, setShowHome] = useState(true);

  return (
    <div>
      <button onClick={() => setShowHome((v) => !v)}>
        {showHome ? "Unmount Home" : "Mount Home"}
      </button>
      {showHome ? <Home /> : <div>Home is hidden</div>}
    </div>
  );
}
```

DÉPART **src/components/Cars.tsx** 96 lignes

```tsx
import { useEffect, useRef, useState } from "react";

```

```
function Car({ make, color }) {
  // useRef garde le compteur d'un render à l'autre, sans provoquer de nouveau render
  const renderCount = useRef(0);
  renderCount.current += 1;
  let colorInfo = color ? color : "Unknown";

  console.log(`Car ${make} Rendered ${renderCount.current} times`, {
    make,
  });

  useEffect(() => {
    console.log(`Car ${make} Mounted`);
    // Fonction de nettoyage : appelée au démontage (bouton « Unmount Home »)
    return () => {
      console.log(`Car ${make} UnMounted`);
    };
  }, [make]);

  return (
    <div className="car-item">
      <div>{make}</div>
      <div>{colorInfo}</div>
    </div>
  );
}

const allCars = [
  { make: "BMW", color: "Blue" },
  { make: "Audi", color: "White" },
  { make: "Mercedes", color: "Black" },
];

function Cars() {
  const renderCount = useRef(0);
  renderCount.current += 1;

  const [cars, setCars] = useState<any>([]);
  const [loading, setLoading] = useState<boolean>(true);

  console.log(`Home Rendered ${renderCount.current} times`, {
    loading,
    cars: cars.length,
  });

  // useEffect avec [] : ça tourne une seule fois, au montage
  useEffect(() => {
    console.log("Home Mounted");
    console.log("Fetching cars...");
    setLoading(true);

    // const timerId =
    setTimeout(() => {
      console.log("Cars fetched");
      setCars(allCars);
      setLoading(false);
    }, 1000);
    return () => {
      console.log("Home UnMounted");
    };
  }, []);

  const handleFilter = (selectedMake: string) => {
    console.log(`Filtering cars by make: ${selectedMake}`);
    if (selectedMake === "ALL") {
      setCars(allCars);
```

```
    } else {
      const filteredCars = allCars.filter((car) => car.make === selectedMake);
      setCars(filteredCars);
      console.log(`Home filtered by make ${selectedMake}`);
    }
  };

  return (
    <div>
      <h1>Welcome to the Home Page</h1>
      <h2>Car List</h2>

      {loading ? (
        <div className="loading">Loading cars...</div>
      ) : (
        <>
          <div className="container">
            {cars.map((car: any, index: number) => (
              <Car key={index} make={car.make} color={car.color} />
            ))}
          </div>
          <button onClick={() => handleFilter("BMW")}>Set Make to BMW</button>
        </>
      )}
    </div>
  );
}

export default Cars;
```

Étape 1 → 2 etape-1-cycle-logs → etape-2-react-memo

React.memo et le state seeMore pour observer les renders inutiles

COMPARÉ `src/components/Cars.tsx → src/components/CarsMemo.tsx` +13 −2

```
@@ -1,3 +1,4 @@
+ import React from "react";
  import { useEffect, useRef, useState } from "react";

  function Car({ make, color }) {
@@ -26,6 +27,9 @@
    );
  }

+ // React.memo : Car ne se re-rend que si ses props changent
+ const MemoizedCar = React.memo(Car);
+
  const allCars = [
    { make: "BMW", color: "Blue" },
    { make: "Audi", color: "White" },
@@ -38,6 +42,7 @@

    const [cars, setCars] = useState<any>([]);
    const [loading, setLoading] = useState<boolean>(true);
+   const [seeMore, setSeeMore] = useState<boolean>(false);

    console.log(`Home Rendered ${renderCount.current} times`, {
      loading,
@@ -74,7 +79,7 @@

    return (
      <div>
-       <h1>Welcome to the Home Page</h1>
+       <h1>{"Welcome to the Home Page"}</h1>
        <h2>Car List</h2>

        {loading ? (
@@ -83,10 +88,16 @@
          <>
            <div className="container">
              {cars.map((car: any, index: number) => (
-               <Car key={index} make={car.make} color={car.color} />
+               <MemoizedCar key={index} make={car.make} color={car.color} />
              ))}
            </div>
            <button onClick={() => handleFilter("BMW")}>Set Make to BMW</button>
+           <button
+             style={{ margin: "10px" }}
+             onClick={() => setSeeMore(!seeMore)}
+           >
+             See More
+           </button>
          </>
        )}
      </div>
```

MODIFIÉ `src/App.jsx` +1 −1

```
@@ -13,7 +13,7 @@
        <button onClick={() => setShowHome((v) => !v)}>
          {showHome ? "Unmount Home" : "Mount Home"}
```

```
15  15          </button>
16      -       {showHome ? <Home /> : <div>Home is hidden</div>}
    16  +       {showHome ? <CarsMemo /> : <div>Home is hidden</div>}
17  17        </div>
18  18      );
19  19    }
```

UTILISÉ **src/components/CarsMemo.tsx** +107 −0

```
@@ -1,0 +1,107 @@
1   + import React from "react";
2   + import { useEffect, useRef, useState } from "react";
3   +
4   + function Car({ make, color }) {
5   +   // useRef garde le compteur d'un render à l'autre, sans provoquer de nouveau render
6   +   const renderCount = useRef(0);
7   +   renderCount.current += 1;
8   +   let colorInfo = color ? color : "Unknown";
9   +
10  +   console.log(`Car ${make} Rendered ${renderCount.current} times`, {
11  +     make,
12  +   });
13  +
14  +   useEffect(() => {
15  +     console.log(`Car ${make} Mounted`);
16  +     // Fonction de nettoyage : appelée au démontage (bouton « Unmount Home »)
17  +     return () => {
18  +       console.log(`Car ${make} UnMounted`);
19  +     };
20  +   }, [make]);
21  +
22  +   return (
23  +     <div className="car-item">
24  +       <div>{make}</div>
25  +       <div>{colorInfo}</div>
26  +     </div>
27  +   );
28  + }
29  +
30  + // React.memo : Car ne se re-rend que si ses props changent
31  + const MemoizedCar = React.memo(Car);
32  +
33  + const allCars = [
34  +   { make: "BMW", color: "Blue" },
35  +   { make: "Audi", color: "White" },
36  +   { make: "Mercedes", color: "Black" },
37  + ];
38  +
39  + function Cars() {
40  +   const renderCount = useRef(0);
41  +   renderCount.current += 1;
42  +
43  +   const [cars, setCars] = useState<any>([]);
44  +   const [loading, setLoading] = useState<boolean>(true);
45  +   const [seeMore, setSeeMore] = useState<boolean>(false);
46  +
47  +   console.log(`Home Rendered ${renderCount.current} times`, {
48  +     loading,
49  +     cars: cars.length,
50  +   });
51  +
52  +   // useEffect avec [] : ça tourne une seule fois, au montage
53  +   useEffect(() => {
54  +     console.log("Home Mounted");
55  +     console.log("Fetching cars...");
```

```
+     setLoading(true);
+
+     // const timerId =
+     setTimeout(() => {
+       console.log("Cars fetched");
+       setCars(allCars);
+       setLoading(false);
+     }, 1000);
+     return () => {
+       console.log("Home UnMounted");
+     };
+   }, []);
+
+   const handleFilter = (selectedMake: string) => {
+     console.log(`Filtering cars by make: ${selectedMake}`);
+     if (selectedMake === "ALL") {
+       setCars(allCars);
+     } else {
+       const filteredCars = allCars.filter((car) => car.make === selectedMake);
+       setCars(filteredCars);
+       console.log(`Home filtered by make ${selectedMake}`);
+     }
+   };
+
+   return (
+     <div>
+       <h1>{"Welcome to the Home Page"}</h1>
+       <h2>Car List</h2>
+
+       {loading ? (
+         <div className="loading">Loading cars...</div>
+       ) : (
+         <>
+           <div className="container">
+             {cars.map((car: any, index: number) => (
+               <MemoizedCar key={index} make={car.make} color={car.color} />
+             ))}
+           </div>
+           <button onClick={() => handleFilter("BMW")}>Set Make to BMW</button>
+           <button
+             style={{ margin: "10px" }}
+             onClick={() => setSeeMore(!seeMore)}
+           >
+             See More
+           </button>
+         </>
+       )}
+     </div>
+   );
+ }
+
+ export default Cars;
```

Étape 2 → 3 etape-2-react-memo → etape-3-usecallback-usememo

useCallback (onSelect) et useMemo (carStyle) pour stabiliser les props

COMPARÉ **src/components/CarsMemo.tsx → src/components/CarsUseMemo.tsx** +38 −12

```
@@ -1,34 +1,38 @@
- import React from "react";
+ import React, { useCallback, useMemo } from "react";
  import { useEffect, useRef, useState } from "react";

- function Car({ make, color }) {
+ function Car({ make, color, onSelect, style }: any) {
    // useRef garde le compteur d'un render à l'autre, sans provoquer de nouveau render
    const renderCount = useRef(0);
    renderCount.current += 1;
-   let colorInfo = color ? color : "Unknown";

-   console.log(`Car ${make} Rendered ${renderCount.current} times`, {
-     make,
-   });
+   console.log(`Car ${make} Rendered ${renderCount.current} times`);

    useEffect(() => {
      console.log(`Car ${make} Mounted`);
      // Fonction de nettoyage : appelée au démontage (bouton « Unmount Home »)
-     return () => {
-       console.log(`Car ${make} UnMounted`);
-     };
+     return () => console.log(`Car ${make} UnMounted`);
    }, [make]);

    return (
-     <div className="car-item">
+     <div className="car-item" style={style} onClick={() => onSelect(make)}>
        <div>{make}</div>
-       <div>{colorInfo}</div>
+       <div>{color ?? "Unknown"}</div>
      </div>
    );
  }

  // React.memo : Car ne se re-rend que si ses props changent
  const MemoizedCar = React.memo(Car);

+
+ /*
+ const MemoizedCar = React.memo(Car, (prevProps, nextProps) => {
+   return (
+     prevProps.brand === nextProps.brand &&
+     prevProps.color === nextProps.color
+   );
+ });
+ */

  const allCars = [
    { make: "BMW", color: "Blue" },
@@ -48,6 +52,22 @@
      loading,
      cars: cars.length,
    });
+
+   // useCallback: même référence entre les renders
+   const onSelect = useCallback((make: string) => {
```

```
+     console.log("Selected:", make);
+   }, []);
+
+   // useMemo: même objet entre les renders
+   const carStyle = useMemo(
+     () => ({
+       padding: "10px",
+       marginBottom: "10px",
+       border: "1px solid #ccc",
+       cursor: "pointer",
+     }),
+     []
+   );
 51  71
 52  72     // useEffect avec [] : ça tourne une seule fois, au montage
 53  73     useEffect(() => {
@@ -88,7 +108,13 @@
 88  108            <>
 89  109              <div className="container">
 90  110                {cars.map((car: any, index: number) => (
-91      -              <MemoizedCar key={index} make={car.make} color={car.color} />
+111 +                <MemoizedCar
+112 +                  key={index}
+113 +                  make={car.make}
+114 +                  color={car.color}
+115 +                  onSelect={onSelect}
+116 +                  style={carStyle}
+117 +                />
 92  118              ))}
 93  119              </div>
 94  120              <button onClick={() => handleFilter("BMW")}>Set Make to BMW</button>
```

MODIFIÉ **src/App.jsx** +1 −1

```
@@ -13,7 +13,7 @@
 13  13         <button onClick={() => setShowHome((v) => !v)}>
 14  14           {showHome ? "Unmount Home" : "Mount Home"}
 15  15         </button>
 16      -       {showHome ? <CarsMemo /> : <div>Home is hidden</div>}
     16 +       {showHome ? <CarsUseMemo /> : <div>Home is hidden</div>}
 17  17       </div>
 18  18     );
 19  19   }
```

UTILISÉ **src/components/CarsUseMemo.tsx** +133 −0

```
@@ -1,0 +1,133 @@
1  + import React, { useCallback, useMemo } from "react";
2  + import { useEffect, useRef, useState } from "react";
3  +
4  + function Car({ make, color, onSelect, style }: any) {
5  +   // useRef garde le compteur d'un render à l'autre, sans provoquer de nouveau render
6  +   const renderCount = useRef(0);
7  +   renderCount.current += 1;
8  +
9  +   console.log(`Car ${make} Rendered ${renderCount.current} times`);
10 +
11 +   useEffect(() => {
12 +     console.log(`Car ${make} Mounted`);
13 +     // Fonction de nettoyage : appelée au démontage (bouton « Unmount Home »)
14 +     return () => console.log(`Car ${make} UnMounted`);
15 +   }, [make]);
16 +
17 +   return (
```

```
+     <div className="car-item" style={style} onClick={() => onSelect(make)}>
+       <div>{make}</div>
+       <div>{color ?? "Unknown"}</div>
+     </div>
+   );
+ }
+
+ // React.memo : Car ne se re-rend que si ses props changent
+ const MemoizedCar = React.memo(Car);
+
+ /*
+ const MemoizedCar = React.memo(Car, (prevProps, nextProps) => {
+   return (
+     prevProps.brand === nextProps.brand &&
+     prevProps.color === nextProps.color
+   );
+ });
+ */
+
+ const allCars = [
+   { make: "BMW", color: "Blue" },
+   { make: "Audi", color: "White" },
+   { make: "Mercedes", color: "Black" },
+ ];
+
+ function Cars() {
+   const renderCount = useRef(0);
+   renderCount.current += 1;
+
+   const [cars, setCars] = useState<any>([]);
+   const [loading, setLoading] = useState<boolean>(true);
+   const [seeMore, setSeeMore] = useState<boolean>(false);
+
+   console.log(`Home Rendered ${renderCount.current} times`, {
+     loading,
+     cars: cars.length,
+   });
+
+   // useCallback: même référence entre les renders
+   const onSelect = useCallback((make: string) => {
+     console.log("Selected:", make);
+   }, []);
+
+   // useMemo: même objet entre les renders
+   const carStyle = useMemo(
+     () => ({
+       padding: "10px",
+       marginBottom: "10px",
+       border: "1px solid #ccc",
+       cursor: "pointer",
+     }),
+     []
+   );
+
+   // useEffect avec [] : ça tourne une seule fois, au montage
+   useEffect(() => {
+     console.log("Home Mounted");
+     console.log("Fetching cars...");
+     setLoading(true);
+
+     // const timerId =
+     setTimeout(() => {
+       console.log("Cars fetched");
+       setCars(allCars);
+       setLoading(false);
```

```
+     }, 1000);
+     return () => {
+       console.log("Home UnMounted");
+     };
+   }, []);
+
+   const handleFilter = (selectedMake: string) => {
+     console.log(`Filtering cars by make: ${selectedMake}`);
+     if (selectedMake === "ALL") {
+       setCars(allCars);
+     } else {
+       const filteredCars = allCars.filter((car) => car.make === selectedMake);
+       setCars(filteredCars);
+       console.log(`Home filtered by make ${selectedMake}`);
+     }
+   };
+
+   return (
+     <div>
+       <h1>{"Welcome to the Home Page"}</h1>
+       <h2>Car List</h2>
+
+       {loading ? (
+         <div className="loading">Loading cars...</div>
+       ) : (
+         <>
+           <div className="container">
+             {cars.map((car: any, index: number) => (
+               <MemoizedCar
+                 key={index}
+                 make={car.make}
+                 color={car.color}
+                 onSelect={onSelect}
+                 style={carStyle}
+               />
+             ))}
+           </div>
+           <button onClick={() => handleFilter("BMW")}>Set Make to BMW</button>
+           <button
+             style={{ margin: "10px" }}
+             onClick={() => setSeeMore(!seeMore)}
+           >
+             See More
+           </button>
+         </>
+       )}
+     </div>
+   );
+ }
+
+ export default Cars;
```